

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TUẦN BẢY
MÔN THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : Kim Ngọc Bách

Sinh viên thực hiện : Nguyễn Trung Nghĩa

Mã Sinh Viên : B23DCCN602

Hà Nội - 2026

1. Mục tiêu hướng đến

Sau khi đã hoàn thiện toàn bộ hệ thống API Backend và kiến trúc lưu trữ ở tuần thứ sáu, tuần thứ bảy của dự án tập trung vào việc hiện thực hóa giao diện người dùng và tích hợp hệ thống. Đây là giai đoạn quan trọng để kết nối các module AI, Database và Security vào một Dashboard trực quan.

Các mục tiêu chính trong tuần này bao gồm:

- Tái cơ cấu toàn bộ cấu trúc thư mục dự án theo chuẩn Mono-repo (Tách biệt BE và FE).
- Xây dựng ứng dụng Frontend sử dụng thư viện React kết hợp với Tailwind CSS.
- Tích hợp luồng livestream nhận diện AI vào giao diện web thời gian thực.
- Xử lý vấn đề đồng bộ hóa dữ liệu giữa các phiên làm việc.
- Hoàn thiện toàn bộ luồng xác thực người dùng: Xác thực Email, Khôi phục mật khẩu.
- Tối ưu hóa hiệu năng ghi dữ liệu vi phạm tại Backend để đảm bảo tính tức thời.

Việc hoàn thành các mục tiêu này giúp hệ thống chuyển từ một backend hoàn chỉnh sang ứng dụng fullstack có khả năng sử dụng thực tế.

2. Tái cấu trúc và Nâng cấp Backend

Để chuẩn bị cho việc tích hợp với Frontend, em đã thực hiện những thay đổi căn bản về mặt kiến trúc Backend nhằm tăng độ ổn định và khả năng mở rộng.

2.1. Tái cơ cấu thư mục và Module Path

Hệ thống được tổ chức lại thành hai thư mục gốc: BE/ (FastAPI) và FE/ (React). Để hỗ trợ phong cách import chuyên nghiệp (SourceCode.BE.app...), em đã cập nhật run.py để tự động nhận diện thư mục gốc thông qua sys.path. Việc này giúp mã nguồn trở nên portable, dễ dàng triển khai trên các môi trường khác nhau.

2.2. Tối ưu hóa cơ chế xác thực cho luồng Livestream MJPEG

Trong quá trình tích hợp luồng video trực tiếp vào giao diện React, em đã đối mặt và xử lý thành công một thách thức lớn về mặt bảo mật và giao thức truyền tải:

- Vấn đề kỹ thuật: Hệ thống sử dụng giao thức MJPEG thông qua `StreamingResponse` để hiển thị video. Cách hiển thị tối ưu nhất trên trình duyệt là sử dụng thẻ `<img>` với thuộc tính `src` trỏ trực tiếp đến endpoint Backend. Tuy nhiên, toàn bộ API của hệ thống đã được bảo vệ nghiêm ngặt bằng cơ chế JWT (JSON Web Token). Do thẻ `<img>` hoạt động theo cơ chế nạp tài nguyên mặc định của trình duyệt, nó không cho phép đính kèm Header tùy chỉnh (Custom Header) như `Authorization: Bearer <token>` giống như các yêu cầu fetch hoặc axios thông thường.
- Giải pháp triển khai: Để vừa đảm bảo tính bảo mật, vừa cho phép hiển thị được video, em đã thực hiện nâng cấp logic xác thực tại Backend:
 - Mở rộng phương thức nhận Token: Sửa đổi Dependency xác thực tại endpoint `/helmet/video-feed` để chấp nhận Access Token thông qua Query Parameter (ví dụ: `/video-feed?token=...`) thay vì chỉ nhận qua Header.
 - Xử lý tại Frontend: Tại component `LiveMonitoring.jsx`, hệ thống sẽ tự động lấy token từ `localStorage`, sau đó nạp vào chuỗi URL của thẻ `<img>`.
- Kết quả: Giải pháp này giúp luồng livestream vẫn được bảo vệ bởi hệ thống Auth (người lạ không có link kèm token đúng sẽ không thể xem lên camera), đồng thời giúp việc tích hợp vào giao diện React trở nên mượt mà, không cần đến các thư viện xử lý stream phức tạp bên thứ ba.

2.3. Tối ưu hóa logic lưu trữ thời gian thực

Trong quá trình tích hợp video livestream, em phát hiện một vấn đề kỹ thuật quan trọng:

- Vấn đề kỹ thuật: `BackgroundTasks` của FastAPI chỉ bắt đầu thực thi sau khi phản hồi kết thúc. Tuy nhiên, luồng Video Stream là một phản hồi vô hạn, dẫn đến việc các vụ vi phạm bị “kẹt” trong hàng đợi và không được lưu vào DB cho đến khi tắt camera.
- Giải pháp triển khai: Em đã thay thế bằng `asyncio.create_task()`. Kỹ thuật này cho phép tạo ra các tác vụ lưu trữ độc lập, chạy song song với luồng stream.

- Kết quả: Dữ liệu vi phạm được đẩy lên MongoDB Atlas và Cloudinary ngay lập tức khi phát hiện, đảm bảo tính cập nhật cho Dashboard.

2.4. Quản lý tài nguyên Camera thông minh

Em đã bổ sung cơ chế phát hiện ngắt kết nối từ phía Client (`request.is_disconnected()`). Khi người dùng tắt tab trình duyệt hoặc chuyển tab trong web, Backend sẽ ngay lập tức nhận biết để giải phóng tài nguyên Camera và dừng mô hình AI, giúp tránh lãng phí GPU và CPU.

3. Xây dựng Dashboard Frontend

Giao diện Dashboard được xây dựng theo phong cách Tactical Dark Mode (Giao diện tối chuyên dụng) nhằm tối ưu hóa khả năng quan sát cho nhân viên an ninh.

3.1. Trang Giám sát trực tiếp (Live Monitoring)

Đây là màn hình quan trọng nhất của hệ thống, tích hợp các công nghệ xử lý trạng thái tiên tiến:

- Session Intelligence: Sử dụng `localStorage` kết hợp với `useRef` để duy trì trạng thái giám sát. Ngay cả khi người dùng F5 hoặc chuyển sang trang khác và quay lại, đồng hồ phiên (Session Time) và các chỉ số thống kê vi phạm vẫn được bảo toàn.
- HUD Overlay: Sử dụng CSS tuyệt đối để vẽ các đường nét trang trí công nghệ và nhãn trạng thái "LIVE" nhấp nháy trên khung hình, tạo cảm giác trực quan sinh động.
- Real-time Alerts: Danh sách cảnh báo bên phải tự động cập nhật và cho phép người dùng click vào ảnh để mở Modal xem chi tiết hoặc tải xuống bằng chứng.

3.2. Trang Lịch sử vi phạm (Violations)

Màn hình này hiện thực hóa toàn bộ các API truy vấn phức tạp của tuần trước:

- Bộ lọc nâng cao: Cho phép lọc theo khoảng thời gian (Date Range), số lượng vi phạm tối thiểu và chế độ "Chỉ hiện vi phạm".

- Modal Navigation: Khi phóng to ảnh bằng chứng, người dùng có thể dùng phím mũi tên hoặc nút bấm để duyệt nhanh qua các vụ vi phạm trước đó/tiếp theo mà không cần đóng cửa sổ.
- Phân trang thông minh: Áp dụng thuật toán phân trang tại Frontend để xử lý mượt mà hàng ngàn bản ghi từ API /violations.

3.3. Trang Phân tích (Analytics)

Đây là module quan trọng giúp chuyển hóa dữ liệu thô từ MongoDB thành thông tin có giá trị vận hành thông qua các kỹ thuật xử lý dữ liệu tại Frontend:

- Logic gộp dữ liệu linh hoạt: Em đã xây dựng cơ chế quản lý độ chi tiết dữ liệu thông qua state. Hệ thống cho phép người dùng chuyển đổi linh hoạt giữa góc nhìn theo Ngày và theo Giờ. Dữ liệu từ API được ánh xạ và chuẩn hóa cấu trúc ngay tại client để phù hợp với định dạng hiển thị của biểu đồ.
- Thuật toán phát hiện đỉnh điểm: Để hỗ trợ người quản lý nhận diện nhanh các "điểm nóng" vi phạm, em sử dụng toán tử spread và phương thức sort để bóc tách bản ghi có lượng vi phạm cao nhất trong tập dữ liệu. Thông tin này được hiển thị nổi bật trên thẻ KPI "Peak Hour".
- Trực quan hóa với thư viện Recharts:
 - Sử dụng AreaChart để biểu diễn xu hướng tổng thể với hiệu ứng gradient mượt mà.
 - Sử dụng BarChart cho các báo cáo định lượng theo giờ để dễ dàng so sánh giữa các mốc thời gian.
 - Hệ thống Tooltip tùy chỉnh cho phép người dùng hover để xem chi tiết cả số lượng vi phạm và độ tin cậy trung bình của AI tại thời điểm đó.

3.4. Trang Quản lý nhân sự (User Management - Admin Only)

Màn hình này được thiết kế dành riêng cho quản trị viên với các ràng buộc bảo mật và logic quản lý dữ liệu chặt chẽ:

- Bộ lọc đa tiêu chí: Em đã xây dựng một hàm lọc phức hợp, cho phép Admin tìm kiếm nhân viên đồng thời theo: ID/Tên đăng nhập/Email, Vai

trò và Trạng thái hoạt động. Việc lọc được thực hiện real-time trên mảng dữ liệu, giúp trải nghiệm quản lý hàng trăm tài khoản trở nên tức thời.

- Cơ chế bảo vệ hệ thống: Để tránh các sai sót vận hành nghiêm trọng, em thiết lập logic chặn hành động xóa hoặc thay đổi trạng thái đối với các tài khoản có quyền Admin. Mọi hành động nhạy cảm khác đều bắt buộc phải qua bước xác nhận để đảm bảo an toàn dữ liệu.
- Quy trình đăng ký tài khoản mới: Form đăng ký được tích hợp ngay trên giao diện thông qua Modal. Toàn bộ dữ liệu được quản lý tập trung qua formData và formErrors, hỗ trợ hiển thị lỗi nghiệp vụ trực tiếp trên từng ô nhập liệu khi Backend trả về thông báo.

3.5. Trang Hồ sơ cá nhân (Profile)

Đây là trang tập trung vào “Technical UX”, đảm bảo tính toàn vẹn dữ liệu và an toàn cho tài khoản người dùng:

- Ánh xạ lỗi từ Backend: Một cải tiến quan trọng là việc xử lý lỗi đồng bộ. Khi Backend trả về mã lỗi 400 hoặc 422 (ví dụ: Username đã tồn tại), thay vì chỉ hiện thông báo chung, hệ thống sẽ tự động bóc tách và gán thông báo lỗi vào đúng ô nhập liệu tương ứng thông qua state errors.
- Xử lý dữ liệu nghiêm ngặt: Trước khi gửi dữ liệu về server, em áp dụng kỹ thuật trim() toàn diện cho tất cả các trường thông tin để loại bỏ khoảng trắng dư thừa, tránh các lỗi logic về định danh.
- Logic bảo mật mật khẩu: Hệ thống thực hiện kiểm tra chéo (new password vs confirm password) ngay tại client. Đồng thời, em bắt các mã lỗi đặc thù như PASSWORD_SAME_AS_OLD từ Backend để đưa ra cảnh báo nhắc nhở người dùng không sử dụng lại mật khẩu cũ.
- Quản lý ảnh đại diện: Tích hợp với Cloudinary API thông qua luồng dữ liệu multipart/form-data. Em sử dụng URL.createObjectURL(file) để tạo ảnh preview tức thời, giúp người dùng nhìn thấy thay đổi ngay lập tức trước khi xác nhận lưu lên đám mây.

4. Hoàn thiện hệ thống Xác thực và Bảo mật

Em đã hoàn thiện các mảnh ghép cuối cùng của hệ thống Auth để biến nó thành một ứng dụng hoàn chỉnh.

- Trang xác thực Email (Verify Email): Tự động bóc tách mã xác thực từ URL query và gọi API Backend ngầm. Giao diện được thiết kế với 3 trạng thái rõ ràng: Đang xác thực, Thành công, và Lỗi.
- Trang khôi phục mật khẩu (Reset Password): Tích hợp tính năng kiểm tra độ mạnh mật khẩu và kiểm tra khớp mật khẩu thời gian thực tại Frontend trước khi gửi dữ liệu về Server.
- Xử lý quên mật khẩu: Tích hợp trực tiếp vào màn hình Login theo dạng Card-flip (chuyển đổi linh hoạt), giúp người dùng không phải tải lại trang khi cần yêu cầu cấp lại mật khẩu.

5. Kết quả đạt được trong tuần 7

Sau khi hoàn thành các công việc trong tuần thứ bảy, các kết quả chính đạt được bao gồm:

- Xây dựng hoàn chỉnh ứng dụng Frontend Dashboard bằng ReactJS.
- Tích hợp thành công toàn bộ API Backend vào giao diện.
- Giải quyết triệt để lỗi “Blocking Background Tasks” trong xử lý Video Stream.
- Triển khai cơ chế lưu trữ bền vững cho dữ liệu phiên làm việc tại Frontend.
- Hoàn thiện luồng logic quên mật khẩu và xác thực tài khoản qua Email một cách chuyên nghiệp.
- Tối ưu hóa UI/UX với các hiệu ứng chuyển cảnh, modal điều hướng và thông báo Toast.

6. Khó khăn gặp phải và cách khắc phục

Trong tuần thứ bảy của dự án, quá trình hiện thực hóa giao diện người dùng và tích hợp toàn diện hệ thống đã đối mặt với những thách thức kỹ thuật phức tạp liên quan đến việc duy trì tính nhất quán của luồng dữ liệu thời gian thực, đồng bộ hóa trạng thái ứng dụng trên môi trường trình duyệt và xử lý các rào cản về giao thức truyền tải đa phương tiện. Các vấn đề này chủ yếu phát sinh từ yêu cầu tối ưu hóa trải nghiệm người dùng (UX) trong một hệ thống giám sát đòi hỏi độ trễ thấp, đồng thời phải đảm bảo tính

bảo mật nghiêm ngặt cho các luồng dữ liệu livestream nhạy cảm. Việc giải quyết các thách thức này đòi hỏi sự phối hợp chặt chẽ giữa logic xử lý bất đồng bộ tại Backend và các kỹ thuật quản lý trạng thái bền vững tại Frontend, nhằm tạo ra một hệ thống vận hành mượt mà và tin cậy.

6.1. Vấn đề “Memory Leak” khi stream video liên tục

Trong quá trình vận hành hệ thống livestream trong thời gian dài, em nhận thấy bộ nhớ RAM trên trình duyệt có xu hướng tăng liên tục, dẫn đến hiện tượng giật lag hoặc thậm chí treo tab.

Nguyên nhân:

- Các frame ảnh JPEG được nạp liên tục nhưng không được giải phóng hoàn toàn
- Việc re-render liên tục trên React gây tích tụ bộ nhớ tạm

Để khắc phục, em đã:

- Tối ưu lại cơ chế render trên React, đảm bảo không giữ reference tới các frame cũ
- Thiết lập cơ chế phát hiện client ngắt kết nối (`request.is_disconnected`) từ phía Backend
- Chủ động dừng luồng stream và giải phóng tài nguyên camera

Kết quả:

- Giảm đáng kể mức tiêu thụ RAM
- Đảm bảo hệ thống hoạt động ổn định trong thời gian dài

6.2. Vấn đề BackgroundTasks không hoạt động với StreamingResponse

Trong quá trình triển khai livestream, em phát hiện rằng BackgroundTasks của FastAPI chỉ được thực thi sau khi response hoàn tất. Tuy nhiên, với StreamingResponse, luồng response kéo dài liên tục, khiến các tác vụ nền không được kích hoạt kịp thời.

Hậu quả:

- Dữ liệu vi phạm không được lưu ngay lập tức
- Giao diện không thể hiển thị realtime các sự kiện vừa xảy ra

Để giải quyết vấn đề này, em đã:

- Thay thế BackgroundTasks bằng `asyncio.create_task`

- Tạo các coroutine chạy song song với luồng streaming

Kết quả:

- Dữ liệu vi phạm được lưu ngay khi phát hiện
- Dashboard có thể cập nhật gần như realtime

6.3. Khó khăn trong việc đồng bộ trạng thái giữa các tab

Ban đầu, các thông tin phiên làm việc như “Số vi phạm”, “Thời gian session” bị reset khi người dùng chuyển giữa các trang, gây gián đoạn trải nghiệm.

Nguyên nhân:

- State được lưu trong React component
- Không có cơ chế lưu trữ bền vững trên trình duyệt

Để khắc phục, em đã:

- Áp dụng cơ chế Persistence State
- Lưu toàn bộ dữ liệu quan trọng vào localStorage
- Đồng bộ state giữa các lần reload hoặc chuyển trang

Kết quả:

- Trạng thái phiên được duy trì ổn định
- Người dùng có trải nghiệm liền mạch hơn khi sử dụng hệ thống

7. Kế hoạch thực hiện trong tuần tiếp theo

Sau khi đã hoàn thiện việc xây dựng và tích hợp hệ thống Frontend với Backend trong tuần thứ bảy, dự án đã tiến gần hơn tới một ứng dụng fullstack hoàn chỉnh có khả năng vận hành thực tế. Tuy nhiên, qua quá trình thử nghiệm (UAT), hệ thống vẫn cần những tinh chỉnh cuối cùng để đạt mức độ hoàn thiện cao hơn về trải nghiệm người dùng và hiệu năng vận hành. Tuần tiếp theo sẽ tập trung vào công tác tối ưu hóa tổng thể, từ giao diện đến khả năng nhận diện, nhằm đảm bảo hệ thống không chỉ ổn định mà còn đạt độ tin cậy cao nhất trước khi bàn giao.

7.1. Tối ưu trải nghiệm người dùng

Mặc dù trong tuần này hệ thống đã hoàn thiện các chức năng cốt lõi như livestream, quản lý vi phạm và xác thực người dùng, tuy nhiên vẫn còn một số thành phần giao diện và luồng tương tác chưa thực sự tối ưu.

Trong tuần tiếp theo, em sẽ tập trung cải thiện trải nghiệm người dùng thông qua:

- Hoàn thiện các thành phần giao diện còn thiếu
- Tối ưu bố cục hiển thị và khả năng tương tác
- Cải thiện tính nhất quán giữa các trang

Việc tối ưu này giúp hệ thống trở nên thân thiện hơn, dễ sử dụng hơn đối với người dùng.

7.2. Cải thiện hiệu năng hệ thống

Mặc dù hệ thống hiện tại đã có thể vận hành ổn định, tuy nhiên trong một số trường hợp, thời gian phản hồi vẫn còn độ trễ nhất định, đặc biệt trong các tác vụ realtime. Nguyên nhân có thể đến từ:

- Tần suất gọi API cao (do sử dụng cơ chế Polling)
- Chưa tối ưu cơ chế truyền dữ liệu realtime

Trong tuần tới, em dự kiến cải thiện hiệu năng thông qua hai hướng kỹ thuật chính:

- Caching dữ liệu: Sử dụng bộ nhớ đệm để giảm tải truy vấn lặp lại cho database.
- WebSockets: Nghiên cứu chuyển đổi cơ chế cập nhật từ Polling sang WebSocket để đảm bảo các thông báo vi phạm được đẩy về giao diện theo thời gian thực (Real-time) đúng nghĩa.

7.3. Tăng cường độ chính xác cho mô hình

Trong quá trình kiểm thử thực tế thông qua webcam trên giao diện web, em nhận thấy độ tin cậy của mô hình còn có sự dao động đáng kể giữa các lần phát hiện.

Nguyên nhân có thể bao gồm:

- Điều kiện ánh sáng không ổn định
- Chất lượng hình ảnh đầu vào thấp (blur, noise)
- Tập dữ liệu huấn luyện chưa đủ đa dạng

Trong tuần tiếp theo, em sẽ thực hiện các hướng cải thiện như:

- Bổ sung và đa dạng hóa dữ liệu huấn luyện
- Tuning lại các tham số của mô hình
- Xem xét áp dụng các kỹ thuật tiền xử lý ảnh

Mục tiêu là nâng cao độ ổn định và độ chính xác của hệ thống nhận diện trong điều kiện thực tế.